
MUHAMMAD TAHA

22F-3277

BSC-3C

4/12/2023

LAB 15

(Total Marks 10)

Instructions:

Lab is viva based only in lab hours.

Use hooking interrupt.

Use enable and disable interrupts on required positions.

Use “in” instruction to read character from keyboard.

Use PIC 60

Task 1: Write a program that counts the number of characters entered by the user until the 'Enter' key is pressed using hardware interrupt. Display the character count on the screen.

```
[org 0x100]
jmp start
count: db 0
oldisr: dd 0 ; space for saving old isr
; keyboard interrupt service routine
kbisr:
    push ax
    push es

    mov ax, 0xb800
    mov es, ax ; point es to video memory

    cmp al, 13      ;enter is down/pressed
    je skip

    in al, 0x60 ; read a char from keyboard port

    xor bx,bx
    mov bx , [count]
    inc bx
```

```

        mov [count], bx

skip:
        jmp nomatch
nomatch:
        ;EOI is no longer needed as the original BIOS routine will have it at its end.
        ;mov al, 0x20
        ;out 0x20, al

print:
        mov bh, 2
        add bl, 0x30
        mov word [es:0], bx

        pop es
        pop ax
        jmp far [cs:oldisr] ; call the original ISR
        ; iret

start:
        xor ax, ax
        mov es, ax ; point es to IVT base
        mov ax, [es:9*4]
        mov [oldisr], ax ; save offset of old routine
        mov ax, [es:9*4+2]
        mov [oldisr+2], ax ; save segment of old routine

        cli ; disable interrupts
        mov word [es:9*4], kbisr ; store offset at n*4
        mov [es:9*4+2], cs ; store segment at n*4+2
        sti ; enable interrupts

l1:
        mov ah, 0 ; service 0 – get keystroke
        int 0x16 ; call BIOS keyboard service
        cmp al, 13 ; is the enter key pressed
        jne l1 ; if no, check for next key

        mov ax, [oldisr] ; read old offset in ax
        mov bx, [oldisr+2] ; read old segment in bx
        cli ; disable interrupts
        mov [es:9*4], ax ; restore old offset from ax
        mov [es:9*4+2], bx ; restore old segment from bx
        sti ; enable interrupts
        mov ax, 0x4c00 ; terminate program
        int 0x21

```



Task 2: Write a program that take string from user and display it in reverse order.

```
[org 0x100]
jmp start
entername: db 'Enter your name'
display:db 'Name:'
oldisr: dd 0 ; space for saving old isr
; keyboard interrupt service routine
kbisr:
    push ax
    push es

    mov ax, 0xb800
    mov es, ax ; point es to video memory

    cmp al, 13      ;enter is down/pressed
    je skip

    in al, 0x60 ; read a char from keyboard port

    mov ah, 0x13 ; service 13 - print string
    mov al, 1 ; subservice 01 – update cursor
    mov bh, 0 ; output on page 0
    mov bl, 04h ; normal attrib
    mov dx, 0x0039 ; row 0 column 39
    mov cx, 5 ; length of string
    push cs
    pop es ; segment of string
    mov bp, display; offset of string
    int 0x10 ; call BIOS video service

    mov di,460
    loooooop:

    mov ah,0x0
    int 0x16

    cmp al,0dh
    je exittt
    mov bl,al
    mov bh,04h

    mov ax, 0xb800
    mov es, ax
    mov word[es:di],bx
    add di, 2
    jmp loooooop

    skip:
```

jmp nomatch

exittt:

nomatch:

;EOI is no longer needed as the original BIOS routine will have it at its end.

;mov al, 0x20

;out 0x20, al

print:

mov bh, 2

add bl, 0x30

mov word [es:0], bx

pop es

pop ax

jmp far [cs:oldisr] ; call the original ISR

; iret

printstr:

push bp

mov bp, sp

push es

push ax

push cx

push si

push di

push ds

pop es ; load ds in es

mov di, [bp+4] ; point di to string

mov cx, 0xffff ; load maximum number in cx

xor al, al ; load a zero in al

repne scasb ; find zero in the string

mov ax, 0xffff ; load maximum number in ax

sub ax, cx ; find change in cx

dec ax ; exclude null from length

jz exit ; no printing if string is empty

mov cx, ax ; load string length in cx

mov ax, 0xb800

mov es, ax ; point es to video base

mov al, 80 ; load al with columns per row

mul byte [bp+8] ; multiply with y position

add ax, [bp+10] ; add x position

shl ax, 1 ; turn into byte offset

mov di, ax ; point di to required location

mov si, [bp+4] ; point si to string

mov ah, [bp+6] ; load attribute in ah

cld ; auto increment mode

nextchar:

```
    lodsb ; load next char in al
    stosw ; print char/attribute pair
    loop nextchar ; repeat for the whole string
```

exit:

```
    pop di
    pop si
    pop cx
    pop ax
    pop es
    pop bp
    ret 8
```

askname:

```
    mov ah, 0x13 ; service 13 - print string
    mov al, 1 ; subservice 01 – update cursor
    mov bh, 0 ; output on page 0
    mov bl, 0x04 ; normal attrib
    mov dx, 0x0A20 ; row 10 column 3
    mov cx, 15; length of string
    push cs
    pop es ; segment of string
    mov bp, entername; offset of string
    int 0x10 ; call BIOS video service
    ret
```

exitt:

```
    ret
```

strReverse:

```
    push bp
    mov bp, sp
    push es
    push ax
    push cx
    push si
    push di
```

```
    mov ax, 0xb800
    mov es, ax
    mov al, 80 ; calculate print position
    mul byte [bp+10]
    add ax, [bp+12]
    shl ax, 1
    mov di, ax ; move location to di to print
    mov si, [bp+6] ; move string start locaton to si
    add si, [bp+4] ; add length to si to reach last of string
    dec si
    mov cx, [bp+4]
```

```

mov ah, [bp+8] ; attribute of string
print_reverse:
std ; reading in reverse
lodsb
cld ; displaying in straight
stosw
loop print_reverse
pop di
pop si
pop cx
pop ax
pop es
pop bp
ret 10
start:
    xor ax, ax
    mov es, ax ; point es to IVT base
    mov ax, [es:9*4]
    mov [oldisr], ax ; save offset of old routine
    mov ax, [es:9*4+2]
    mov [oldisr+2], ax ; save segment of old routine

    cli ; disable interrupts
    mov word [es:9*4], kbisr ; store offset at n*4
    mov [es:9*4+2], cs ; store segment at n*4+2
    sti ; enable interrupts

l1:
    mov ah, 0 ; service 0 – get keystroke
    int 0x16 ; call BIOS keyboard service
    cmp al, 13 ; is the enter key pressed
    jne l1 ; if no, check for next key

    mov ax, [oldisr] ; read old offset in ax
    mov bx, [oldisr+2] ; read old segment in bx
    cli ; disable interrupts
    mov [es:9*4], ax ; restore old offset from ax
    mov [es:9*4+2], bx ; restore old segment from bx
    sti ; enable interrupts

    mov ax, display
    push ax
    mov ax, 5 ; length reversed string
    push ax
    call strReverse

    mov ax, 0x4c00 ; terminate program
    int 0x21

```

Best of luck

